

Auto Git Pull via GitHub Webhook

Automatically update your server when new code is pushed to GitHub using:

- GitHub Webhooks
- A local webhook listener
- A deployment script

Why Use GitHub Webhooks?

Without webhooks:

- You push to GitHub
- Then **manually** SSH into server
- Run `git pull`, rebuild, and restart

With webhooks:

- ✓ The server is updated **automatically**
- ✓ No manual deployment needed
- ✓ Great for continuous delivery

How It Works

1. GitHub triggers a webhook when code is pushed
2. Your server listens for the webhook
3. A script runs:
 - Pulls latest code
 - Rebuilds Docker image
 - Restarts the app

Step 1: Install **webhook** Listener

Download precompiled binary:

```
wget https://github.com/adnanh/webhook/releases/latest/download/webhook-linux-amd64.tar.gz
tar -xzf webhook-linux-amd64.tar.gz
sudo mv webhook /usr/local/bin/
```

Step 2: Create Hook Configuration

hooks.json

```
[
  {
    "id": "flask-hook",
    "execute-command": "/home/deploy/deploy.sh",
    "command-working-directory": "/home/deploy/my-flask-app"
  }
]
```

Step 3: Start Webhook Listener

```
webhook -hooks hooks.json -port 9000
```

Your server now listens at:

```
http://<server-ip>:9000/hooks/flask-hook
```

Step 4: Set GitHub Webhook

1. Go to your repo → **Settings** → **Webhooks**

2. Add webhook:

- **Payload URL:** `http://your-server:9000/hooks/flask-hook`
- **Content type:** `application/json`
- Optional: add a secret

3. Save and test

Step 5: Test It

1. Make a commit and push to `main`
2. GitHub will hit your webhook endpoint
3. Your script runs:
 - `git pull`
 - `docker build`
 - `docker restart`

 The app updates itself!

Common Pitfalls

Issue	Fix
Port 9000 blocked	Allow it in firewall
Webhook doesn't trigger	Check delivery logs in GitHub
Permission errors	Ensure script is executable and user has Docker access

Summary

- Webhooks automate deployments on code push
- Simple listener runs a shell script
- Combine with GitHub Actions for full CI/CD

 **Hands-Free Deployment Achieved!**